

cryptography escape room

Younes Menfalouti

Warm up

```
CryptographyLab > Caesar_cipher.py > Caesar_decipher
1
2 def Caesar_cipher(text, shift):
3     result = ""
4     base: int = ord('A')
5
6     for char in text:
7         if char == ' ':
8             result += ' '
9         else:
10            result += chr((ord(char) - base + shift) % 26 + base)
11
12    return result
13
14 def Caesar_decipher(text, shift):
15     result = ""
16     base: int = ord('A')
17
18     for char in text:
19         if char == ' ':
20             result += ' '
21         else:
22            result += chr((ord(char) - base - shift) % 26 + base)
23
24    return result
25
26 print("CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ encrypted : ", Caesar_cipher("CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ", 4))
27
28 print("GWCL JMBBMZ VNB JM CAQVO BWNTA BW AWTDM BPQA decrypted : ", Caesar_decipher("GWCL JMBBMZ VNB JM CAQVO BWNTA BW AWTDM BPQA", 8))
29
30 for i in range(1, 26):
31     print(f"Shift {i}: {Caesar_decipher('ESP VPJ TD FYVZYH ECJ MCFEP QZCNP', i)}")
32
```

Figure 1. Caesar encryption, decryption and brute force.

With $K = 4$ the message becomes GVCTXS MW JYR YRXMP XLI TVSJIIWWSV WECW UYMD. With $K = 8$ the second message decrypts to YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS.

```
CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ encrypted :  GVCTXS MW JYR YRXMP XLI TVSJIIWWSV WECW UYMD
GWCL JMBBMZ VNB JM CAQVO BWNTA BW AWTDM BPQA decrypted :  YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS
```

Figure 2. Encryption and decryption results.

Caesar has only 25 useful keys, so trying them all is instant. Shift 11 is the only readable line and gives THE KEY IS UNKNOWN TRY BRUTE FORCE.

```
Shift 2: CQN TNH RB DWTWFW CAH KADCN OXALN
Shift 3: BPM SMG QA CWSVWEV BZG JZCBM NWZKM
Shift 4: AOL RLF PZ BURUVDU AYF IYBAL MYYJL
Shift 5: ZNK QKE OY ATQTUCT ZXE HXAZJ LUXIK
Shift 6: YMJ PJD NX ZSPSTBS YWD GWZYJ KTWJH
Shift 7: XLI OIC MM YRORSAR XVC FVYXI JSVGI
Shift 8: WKH NHB LV XQNQRZQ WUB EUXWH IRUFH
Shift 9: VJG MGA KU WPMPQYP VTA DTWVG HQTEG
Shift 10: UIF LFZ JT VOLOPXO USZ CSVUF GPSDF
Shift 11: THE KEY IS UNKNOWN TRY BRUTE FORCE
Shift 12: SGD JDX HR TMJMNVM SQX AQTSD ENQBD
Shift 13: RFC ICW GQ SLILMUL RPW ZPSRC DMPAC
Shift 14: QEB HBV FP RKHKLTQ QOV YORQB CLOZB
Shift 15: PDA GAU EO QJGJSJ PNU XNQA BKNYA
Shift 16: OCZ FZT DN PIFIJRI OMT WMPQZ AJMXZ
Shift 17: NBY EYS CM OHEHIQH NLS VLONY ZILWY
Shift 18: MAX DXR BL NGDGHGP MKR UKNMX YHKVX
Shift 19: LZW CWQ AK MFCFGOF LJQ TJMLW XGJUW
Shift 20: KYV BVP ZJ LEBEFNE KIP SILKV WFITV
Shift 21: JXU AUO YI KDADEMD JHO RHKJU VEHSU
Shift 22: IWT ZTN XH JCZCDLC IGN QGJIT UDGRT
Shift 23: HVS YSM WG IBYBCKB HFM PFIHS TCFQS
Shift 24: GUR XRL VF HAXABJA GEL OEHGR SBEPR
```

Figure 3. All Caesar keys tried on the third message.

Room 1, the letter wall

```
CryptographyLab > Sub_cypher.py > Substitution
1 def Sub_cypher(text, sub):
2
3     encrypted_text = ""
4     for char in text:
5         if char in sub:
6             encrypted_text += sub[char]
7         else:
8             encrypted_text += char
9     return encrypted_text
10
11
12 def Sub_decipher(text, sub):
13     decrypted_text = ""
14     reverse_sub: dict[Unknown, Unknown] = {v: k for k, v in sub.items()}
15     for char in text:
16         if char in reverse_sub:
17             decrypted_text += reverse_sub[char]
18         else:
19             decrypted_text += char
20     return decrypted_text
21
22 def Analyze(text) :
23     frequency: dict[Unknown, Unknown] = {}
24     sum = 0
25     for char in text:
26         if char.isalpha():
27             char = char.upper()
28             frequency[char] = frequency.get(char, 0) + 1
29             sum += 1
30
31     for freq in frequency:
32         frequency[freq] = (frequency[freq] / sum) * 100
33
34     return frequency
35
```

Figure 4. Substitution and frequency analysis functions.

```

37 substitution_dict = {}
38 'A': 'Q',
39 'B': 'M',
40 'C': 'J',
41 'D': 'Z',
42 'E': 'T',
43 'F': 'S',
44 'G': 'P',
45 'H': 'K',
46 'I': 'R',
47 'J': 'N',
48 'K': 'L',
49 'L': 'S',
50 'M': 'B',
51 'N': 'O',
52 'O': 'X',
53 'P': 'H',
54 'Q': 'C',
55 'R': 'E',
56 'S': 'F',
57 'T': 'A',
58 'U': 'V',
59 'V': 'W',
60 'W': 'I',
61 'X': 'A',
62 'Y': 'D',
63 'Z': 'G'
64
65 print("KEEP THE SECRET SAFE encrypted : ", Sub_cipher("KEEP THE SECRET SAFE", substitution))
66 print("\n")
67 print("ERVYE DXVR SXFPJ decrypted : ", Sub_decipher("ERVYE DXVR SXFPJ", substitution))
68 print("\n")
69 Analyzed_frequency = Analyze("EKT YTJRTE BTYVQT PY KPZTTO PO EKPV NQRFQW. STEETH GRVCYVD ZXY OX KTHQS EXT IXST LTD.WE PE FPNY Q")
70 print("Analyzed frequency of the text: ", sorted(Analyzed_frequency.items(), key=lambda x: x[1], reverse=True))
71
72
73
74
75
76
77

```

```

78 substitution_dict = {}
79 'T': 'E',
80 'M': 'K',
81 'E': 'T',
82 'A': 'Q',
83 'J': 'P',
84 'S': 'F',
85 'C': 'J',
86 'R': 'E',
87 'D': 'Z',
88 'L': 'S',
89 'O': 'X',
90 'B': 'M',
91 'N': 'O',
92 'P': 'H',
93 'Q': 'C',
94 'U': 'V',
95 'V': 'W',
96 'W': 'I',
97 'I': 'R',
98 'H': 'K',
99 'G': 'P',
100 'K': 'L',
101 'X': 'A',
102 'Y': 'D',
103 'A': 'A',
104 'Z': 'G'
105
106
107
108 print("\n")
109 print("Decrypted text using the analyzed frequency: ", Sub_decipher("EKT YTJRTE BTYVQT PY KPZTTO PO EKPV NQRFQW. STEETH GRVCYVD ZXY")

```

Figure 5. The given alphabet and the recovered key.

With the given alphabet **KEEP THE SECRET SAFE** becomes **LTTN EKT YTJRTE YQGT** and **ERVYE DXVR SXFPJ** decrypts to **TRUST YOUR LOGIC**.

A substitution key has $26!$ possible values, so brute force is impractical. I instead matched the most frequent ciphertext letters with the most frequent English letters, then fixed the rest with short words and repeated patterns. Here **T** appears 17.3% of the time and maps to **E**, then **E** maps to **T** and **R** to **R**. Words like **EKT** for **THE** complete the key, which turns out to be the same alphabet as in the first tasks.

```

KEEP THE SECRET SAFE encrypted : LTTN EKT YTJRTE YQGT
=====
ERVYE DXVR SXFPJ decrypted : TRUST YOUR LOGIC
=====
Analyzed frequency of the text: [('T', 17.258883248730964), ('E', 9.644670050761421), ('R', 9.137055837563452),
('Y', 8.121827411167512), ('X', 8.121827411167512), ('Q', 6.091370558375635), ('K', 5.583756345177665), ('O', 4.568527918781726),
('Z', 4.060913705583756), ('P', 3.5532994923857872), ('F', 3.0456852791878175), ('S', 2.5380710659898478),
('J', 2.030456852791878), ('B', 2.030456852791878), ('N', 2.030456852791878), ('V', 2.030456852791878), ('D', 2.030456852791878),
('H', 2.030456852791878), ('I', 2.030456852791878), ('G', 1.5228426395939088), ('L', 1.5228426395939088), ('C', 0.5076142131979695), ('M', 0.5076142131979695)]
=====
Decrypted text using the analyzed frequency: THE SECRET MESSAGE IS HIDDEN IN THIS PARAGRAPH. LETTER FREQUENCY DOES NOT REVEAL THE WHOLE KEY, BUT IT GIVES A GOOD START. LOOK FOR COMMON WORDS, REPEATED PATTERNS, AND SHORT WORDS. WHEN YOU RECOVER THE MESSAGE, THE VIGENERE KEY HAS FOUR LETTERS.
❖➔ Information-Security git:(main) ✖

```

Figure 6. Frequencies and decrypted message.

The message gives the clue for the next room, the Vigenère key has four letters.

Room 2, the Vigenère cipher

```
CryptographyLab > Vigenere_cypher.py > ...
1 from Sub_cypher import Analyze
2
3 def Vigenere_key_finder(text, key_len : int):
4     groups: list[Unknown] = [text[i::key_len] for i in range(key_len)]
5     frequencies: list[Unknown] = [Analyze(group) for group in groups]
6     key = ""
7     for freq in frequencies:
8         most_common = max(freq, key=freq.get)
9         shift: int = (ord(most_common) - ord('E')) % 26
10        key += chr(shift + ord('A'))
11    return key
12
13 def Vigenere_decipher(text, key):
14     decrypted_text = ""
15     key_length: int = len(key)
16     key_index = 0
17
18     for char in text:
19         if char.isalpha():
20             shift: int = ord(key[key_index % key_length]) - ord('A')
21             decrypted_char: str = chr((ord(char) - ord('A') - shift) % 26 + ord('A'))
22             decrypted_text += decrypted_char
23             key_index += 1
24         else:
25             decrypted_text += char
26
27    return decrypted_text
28
29 key = Vigenere_key_finder("VVHWGQUIVYHIRSUPGTWXJSVIPCIUHPGCIIVVHPGHWITSOTRSDVUSYITMZLGFHIXSUCVSDQSPFGFVLQIOHEVHGMSDGJQRPAQWGSNXJSPSUHI
30 print("Key is : ", key)
31
32 print('=' * 50)
33
34 print("Decrypted text is : ", Vigenere_decipher("VVHWGQUIVYHIRSUPGTWXJSVIPCIUHPGCIIVVHPGHWITSOTRSDVUSYITMZLGFHIXSUCVSDQSPFGFVLQIOHEVHGMS
35
```

Figure 7. Key recovery and Vigenère decryption.

The ciphertext is split into four groups, one per key letter, and each group is a Caesar cipher. The most frequent letter of each group is taken as E, which gives the shifts 2, 14, 3 and 4 and the key CODE.

```
Key is : CODE
=====
Decrypted text is : THESECRETKEEPERLEFTTHESENOTESBECAUSETHELETTEREAPPEARSEVERYWHEREEVERYTEAMMEMBERSHOULDCHECKEA
CHCOLUMNSEEKTHEMOSTFREQUENTSMBOLANDUSEITTORECOVERTHEKEYWHENTHEMESSAGEBECOMESCLEARENTERTHEACCESSWORDCLIPSE
Information Security city (main) ✓
```

Figure 8. Recovered key and plaintext.

The plaintext ends with ENTER THE ACCESS WORD ECLIPSE, so the access word is ECLIPSE.

Conclusion

Caesar falls to brute force because its key space is tiny. Substitution and Vigenère have large key spaces but keep the letter frequencies of English, in the whole text or in each column, so frequency analysis breaks them.